МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМ. Р.Е. АЛЕКСЕЕВА»



Институт радиоэлектроники и информационных технологий
Кафедра «Компьютерные технологии в проектировании и производстве»

Отчет по лабораторной работе №2

Исследование алгоритма сжатия Move-to-Front («стопка книг»)
и префиксного кодирования текстовых данных

по дисциплине «Основы радиоэлектроники и связи»

СТУДЕНТ

	Разумов В.А., Стенин Д.Ю.
(подпись)	*(фамилия, и., о.)*
	23-ТР
(дата)	*(группа или шифр)*

ПРЕПОДАВАТЕЛЬ

	Штанюк А.А.
(подпись)	*(фамилия, и., о.)*
	(дата)

Комментарии	*Дата сдачи*	*Дата проверки*	*Отметка о выполнении*

Нижний Новгород 2026 г.

Копировал Формат А4

Содержание

Введение 3
1 Теоретические основы 4
1.1 Алгоритм Move-to-Front 4
1.2 Префиксный унарный код 4
1.3 Оценка сжатия Deflate 4
2 Ход работы 5
3 Листинг программы 6
4 Итоги работы программы 8
4.1 Анализ полученных результатов 8
5 Выводы 9

Введение

В трактах передачи цифровых данных эффективность использования канала связи определяется объемом избыточности сообщений. Естественный текст обладает локальной контекстной связностью: близлежащие слова строятся из ограниченного набора повторяющихся букв.

Алгоритм Move-to-Front (MTF, «стопка книг») устраняет контекстную избыточность динамически перемещая недавно использованные символы в начало списка. Входной поток преобразуется в малые целые числа. В комбинации с префиксным кодированием это уменьшает число передаваемых двоичных символов и повышает степень сжатия архиваторами.

Цель работы: исследование эффективности алгоритма Move-to-Front и префиксного кодирования на примере текста романа «Война и мир», расчет числа передаваемых бит и оценка сжатия алгоритмом Deflate.

Задачи работы:

1. адаптировать алгоритм MTF для текста романа «Война и мир»;
2. выбрать префиксный код и закодировать содержимое файла;
3. определить число символов („0“, „1“), передаваемых по каналу связи при прямом кодировании и с MTF;
4. сжать исходный и закодированный файлы архиватором ZIP и сравнить результаты;
5. проанализировать полученные характеристики.

1 Теоретические основы

1.1 Алгоритм Move-to-Front

Алгоритм MTF оперирует упорядоченным списком символов алфавита $A = (a_0, a_1, ..., a_{m-1})$ мощностью m.

При считывании очередного символа находится его текущий индекс $i \in [0, m-1]$ в списке. Значение i выводится на выход, после чего символ переносится на нулевую позицию со сдвигом остальных элементов вправо.

В силу контекстной повторяемости слов символы кодируются малыми индексами $(0, 1, 2, ...)$. Выходной поток получает монотонно спадающее распределение вероятностей.

1.2 Префиксный унарный код

Для кодирования номеров выбран унарный префиксный код:

$$C(i) = \underbrace{11...1}_{i}0. \tag{1.1}$$

Длина кодового слова для индекса i:

$$L(i) = i + 1 \quad [\text{бит}]. \tag{1.2}$$

Символ `0` является маркером окончания слова, что обеспечивает однозначное префиксное декодирование. Малые индексы получают минимальную разрядность: $C(0) = 0$, $C(1) = 10$, $C(2) = 110$.

1.3 Оценка сжатия Deflate

Для проверки сжимаемости использовался алгоритм Deflate (ZIP), объединяющий LZ77 и код Хаффмана. Коэффициент сжатия:

$$K_{\text{сжат}} = \frac{V_{\text{исх}}}{V_{\text{zip}}}. \tag{1.3}$$

2 Ход работы

1. Был взят текст романа «Война и мир» на русском языке (`war_peace_rus.txt`) объемом 1 908 695 байт ($m = 137$ символов).
2. Были реализованы функции `mft_encode`, `mft_decode` и `compress_to_zip`.
3. Сформирован массив унарных кодов `unary_dict`.
4. Выполнено MTF-кодирование текста романа. Индексы преобразованы в унарную битовую строку и записаны в `war_peace_rus.mft`.
5. Выполнено прямое унарное кодирование текста без MTF.
6. Подсчитано суммарное количество символов `'0'` и `'1'` для обоих вариантов передачи.
7. Исходный и закодированный файлы сжаты архиватором ZIP (`compresslevel = 9`). Вычислены коэффициенты сжатия.

3 Листинг программы

Ниже приведен исходный код функций (Листинг 3.1) и сценария расчета (Листинг 3.2).

```python
import requests
import zipfile
from pathlib import Path
from typing import Iterable

def mft_encode(data: str, alphabet: Iterable[str]) -> list[int]:
    alphabet = list(alphabet)
    output = []
    for char in data:
        idx = alphabet.index(char)
        output.append(idx)
        alphabet.insert(0, alphabet.pop(idx))
    return output

def mft_decode(data: list[int], alphabet: Iterable[str]) -> str:
    alphabet = list(alphabet)
    output = ""
    for idx in data:
        output += alphabet[idx]
        alphabet.insert(0, alphabet.pop(idx))
    return output

def compress_to_zip(
    input_file: Path,
    output_zip: Path,
    arcname: str | None = None,
    compresslevel: int = 9,
) -> int:
    if arcname is None:
        arcname = input_file.name

    with zipfile.ZipFile(
        output_zip,
        mode="w",
        compression=zipfile.ZIP_DEFLATED,
        compresslevel=compresslevel,
    ) as zf:
        zf.write(input_file, arcname=arcname)

    return output_zip.stat().st_size
```

Листинг 3.1 — Функции алгоритма Move-to-Front и компрессии в формат ZIP

```python
file_path = Path("war_peace_rus.txt")
if not file_path.exists():
    war_peace = requests.get(
        "https://raw.githubusercontent.com/NNTU-CS/RECF-2026/refs/heads/main/Data/war_peace_rus.txt"
    )
    file_content = war_peace.content.decode("utf8")
    file_path.write_text(file_content)
else:
    file_content = file_path.read_text()

alphabet = list(set(file_content))
unary_dict = [("1" * i) + "0" for i in range(len(alphabet))]

file_encoded = mft_encode(file_content, alphabet)
encoded_unary = "".join([unary_dict[idx] for idx in file_encoded])

encoded_path = Path("war_peace_rus.mft")
encoded_path.write_text(encoded_unary)

content_unary = "".join([
    unary_dict[idx]
    for idx in map(alphabet.index, file_content)
])

print(f"Original / encoded length: {len(content_unary)} / {len(encoded_unary)} "
      f"({len(content_unary) / len(encoded_unary):.3f})")

compressed_path = Path("war_peace_rus.zip")
compressed_size = compress_to_zip(file_path, compressed_path)

compressed_encoded_path = Path("war_peace_rus.mft.zip")
compressed_encoded_size = compress_to_zip(encoded_path, compressed_encoded_path)

print(f"Encoded compressed / compressed size: {compressed_encoded_size} / {compressed_size} "
      f"({compressed_encoded_size / compressed_size:.3f})")
```

Листинг 3.2 — Сценарий кодирования текста романа и замера объемов данных

4 Итоги работы программы

Результаты эксперимента сведены в Таблица 4.1.

Таблица 4.1 — Результаты кодирования и сжатия романа «Война и мир»

Параметр	**Без MTF (исходный)**	**С алгоритмом MTF**	**Соотношение / Выигрыш**
Длина текста N, символов	1 908 695	1 908 695	1.000
Мощность алфавита m	137	137	—
Бит в канале связи („0“, „1“)	146 319 495	26 158 085	**в 5.594 раза** (экономия 82.1%)
Средняя длина на символ	76.66 бит	13.71 бит	уменьшение в 5.594 раза
Размер файла на диске	1 908 695 байт	26 158 085 байт	файл `.mft` (ASCII-символы)
Размер ZIP-архива (Deflate)	753 130 байт	1 696 840 байт	2.253
Коэффициент сжатия $K_{\text{сжат}}$	**2.534**	**15.416**	**рост в 6.08 раза**

4.1 Анализ полученных результатов

Прямое унарное кодирование текста без предварительной обработки требует передачи $146\,319\,495$ бит по каналу связи. Редкие символы алфавита получают длинные кодовые слова из десятков единиц.

Применение алгоритма MTF снизило объем передаваемых бит до $26\,158\,085$. Получен выигрыш в 5.594 раза так как основная масса символов закодирована первыми индексами стопки.

При сжатии архиватором Deflate исходный текст показал коэффициент $K_{\text{исх}} = 2.534$. Закодированный файл `war_peace_rus.mft` был сжат с коэффициентом $K_{\text{mft}} = 15.416$ (в 6.08 раза выше). Высокая компрессия обусловлена тем, что унарная последовательность после MTF состоит из длинных серий „1“ и редких „0“, эффективно сжимаемых алгоритмом LZ77.

5 Выводы

1. Проведена адаптация алгоритма Move-to-Front для русского текста романа «Война и мир» объемом 1.91 млн символов.
2. В качестве префиксного кода выбран унарный код $C(i) = 1^i0$, обеспечивающий однозначное декодирование.
3. Алгоритм MTF снижает число передаваемых по каналу двоичных символов со 146 319 495 до 26 158 085 бит (выигрыш в 5.594 раза или 82.1%).
4. Коэффициент сжатия Deflate для MTF-файла составил 15.416 против 2.534 для исходного текста, что подтверждает эффективность применения MTF перед словарными алгоритмами.